Consultas de Auditoria — Petly Club

ERP Petly Club · Grupo 7 · Modelagem de Dados · PostgreSQL 16 · executadas em 27/10/2026

> **Responsabilidade por esta parte:** na divisão da entrega final, as consultas de auditoria eram responsabilidade do integrante **Amin**. Ele não entregou esta parte e não teve nenhum tipo de participação no projeto depois da primeira entrega. Por causa dessa falta de colaboração, a parte foi feita por **Gabriela Colombo**, com auxílio de IA (Claude). Todas as consultas foram executadas e conferidas no banco do grupo, como mostram os resultados abaixo.

Objetivo

Uma consulta de auditoria não responde a uma pergunta de venda: ela procura no banco situações que merecem verificação. As restrições e os gatilhos do DDL já impedem muitos erros, como CPF repetido e venda sem saldo (ver os testes de integridade). Estas consultas cobrem o que o banco **aceita**, mas o negócio não deveria ter: regras que dependem de várias tabelas, do tempo ou de permissões, que o Modelo Lógico deixa para a aplicação (seção 6).

> **Como foram testadas:** cada consulta foi executada duas vezes no PostgreSQL 16, sobre a base criada com `petly_club_ddl_limpo.sql` e `Massa_de_Dados_Completa_-_ERP_Petly_Club.sql`.
> 1. **Na base do grupo:** o resultado esperado é nenhuma linha, porque a massa de dados foi conferida e está consistente.
> 2. **Com uma inconsistência proposital:** dentro de BEGIN … ROLLBACK, inserimos um erro que as restrições aceitam e rodamos a consulta de novo. Ela precisa encontrar exatamente esse erro. No final, o ROLLBACK desfaz tudo.
> Os comandos estão em `consultas_auditoria.sql`.

Resumo

Consulta	O que procura	Regra	Na base do grupo	Com o erro proposital
AU01	Pedidos concluídos sem pagamento suficiente	RN15, RN16	0 linhas	**Encontrou ✔**
AU02	Pedidos sem itens	RN03	0 linhas	**Encontrou ✔**
AU03	Itens vendidos sem baixa correta no estoque	RN20, RN21	0 linhas	**Encontrou ✔**
AU04	Reservas presas em pedidos encerrados	RN21, RN22	0 linhas	**Encontrou ✔**
AU05	Pagamentos e contas a receber que não batem	RN22, RN24	0 linhas	**Encontrou ✔**
AU06	Compras com recebimento ou financeiro inconsistente	RN12, RN25	0 linhas	**Encontrou ✔**
AU07	Lotes vencidos ainda com saldo	RF12, RP09	0 linhas	**Encontrou ✔**
AU08	Perdas e ajustes autorizados por quem não é gerente	RP08, RP11	0 linhas	**Encontrou ✔**
AU09	Descontos sem autorização registrada	RP05, RF32	0 linhas	**Encontrou ✔**
AU10	Cadastros de clientes possivelmente duplicados	RN18, RF03	0 linhas	**Encontrou ✔**

Resultado geral: as 10 consultas não encontraram nenhuma inconsistência na base do grupo e encontraram cada erro inserido de propósito.

Consultas

[AU01] Pedidos concluídos sem pagamento suficiente

O que verifica: Procura pedidos já concluídos (pago, enviado, entregue ou retirado) em que a soma dos pagamentos autorizados é diferente do valor total do pedido. O banco não impede isso sozinho: a conferência fica na aplicação ([TRG-11]).

Regra: RN15, RN16

SQL:

```
SELECT p.id_pedido, p.canal_venda, p.status,
       t.valor_total,
       COALESCE(SUM(g.valor) FILTER (WHERE g.status = 'autorizado'), 0) AS pago_autorizado
FROM pedido p
JOIN v_pedido_total t ON t.id_pedido = p.id_pedido
LEFT JOIN pagamento g ON g.id_pedido = p.id_pedido
WHERE p.status IN ('pago', 'enviado', 'entregue', 'retirado')
GROUP BY p.id_pedido, p.canal_venda, p.status, t.valor_total
HAVING COALESCE(SUM(g.valor) FILTER (WHERE g.status = 'autorizado'), 0) <> t.valor_total
ORDER BY p.id_pedido;
```

Resultado na base do grupo: nenhuma linha.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
-- Pedidos concluídos sem pagamento suficiente (base do grupo)
 id_pedido | canal_venda | status | valor_total | pago_autorizado
-----------+-------------+--------+-------------+-----------------
(0 rows)
```

Teste com erro proposital: Um pedido marcado como 'pago' sem nenhum pagamento registrado.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO pedido (id_pedido, canal_venda, tipo_recebimento, status, id_cliente, id_funcionario) OVERRIDING SYSTEM VALUE
petly_club-# VALUES (90, 'loja_fisica', 'balcao', 'pago', 1, 2);
INSERT 0 1
petly_club=# INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario) VALUES (90, 5, 1, 74.90);
INSERT 0 1
-- consulta AU01 depois da inconsistência:
 id_pedido | canal_venda | status | valor_total | pago_autorizado
-----------+-------------+--------+-------------+-----------------
        90 | loja_fisica | pago   |       74.90 |               0
(1 row)

(ROLLBACK: a base volta ao estado original)
```

Decisão: Conferir o caixa do dia e o pagamento com o cliente ou com a operadora antes do fechamento (RF26).

[AU02] Pedidos sem itens

O que verifica: Procura pedidos que não têm nenhum produto. Todo pedido precisa de pelo menos um item, mas como o item é que aponta para o pedido, uma FK não garante isso ([TRG-10]).

Regra: RN03

SQL:

```
SELECT p.id_pedido, p.data_pedido, p.canal_venda, p.status
FROM pedido p
WHERE NOT EXISTS (SELECT 1 FROM item_pedido i WHERE i.id_pedido = p.id_pedido)
ORDER BY p.id_pedido;
```

Resultado na base do grupo: nenhuma linha.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
-- Pedidos sem itens (base do grupo)
 id_pedido | data_pedido | canal_venda | status
-----------+-------------+-------------+--------
(0 rows)
```

Teste com erro proposital: Um pedido aberto na loja virtual sem nenhum produto.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO pedido (id_pedido, canal_venda, tipo_recebimento, id_cliente) OVERRIDING SYSTEM VALUE
petly_club-# VALUES (91, 'loja_virtual', 'entrega', 2);
INSERT 0 1
-- consulta AU02 depois da inconsistência:
 id_pedido |        data_pedido         | canal_venda  |  status
-----------+----------------------------+--------------+----------
        91 | 2026-10-27 23:51:33.473256 | loja_virtual | pendente
(1 row)

(ROLLBACK: a base volta ao estado original)
```

Decisão: Cancelar ou completar o pedido; pedido vazio distorce os relatórios de vendas e o ticket médio.

[AU03] Itens vendidos sem baixa correta no estoque

O que verifica: Compara, em cada item de pedido concluído, a quantidade vendida com o que saiu dos lotes (vendas menos devoluções). Se for diferente, o estoque físico não bate com as vendas.

Regra: RN20, RN21

SQL:

```
SELECT i.id_pedido, i.id_produto, i.quantidade AS vendido,
       COALESCE(SUM(CASE WHEN m.tipo = 'venda' THEN m.quantidade
                         WHEN m.tipo = 'devolucao' THEN -m.quantidade END), 0) AS baixado
FROM item_pedido i
JOIN pedido p ON p.id_pedido = i.id_pedido
LEFT JOIN movimentacao_estoque m ON m.id_pedido = i.id_pedido AND m.id_produto = i.id_produto
WHERE p.status IN ('pago', 'enviado', 'entregue', 'retirado')
GROUP BY i.id_pedido, i.id_produto, i.quantidade
HAVING COALESCE(SUM(CASE WHEN m.tipo = 'venda' THEN m.quantidade
                         WHEN m.tipo = 'devolucao' THEN -m.quantidade END), 0) <> i.quantidade
ORDER BY i.id_pedido, i.id_produto;
```

Resultado na base do grupo: nenhuma linha.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
-- Itens vendidos sem baixa correta no estoque (base do grupo)
 id_pedido | id_produto | vendido | baixado
-----------+------------+---------+---------
(0 rows)
```

Teste com erro proposital: O pedido vendeu 3 shampoos, mas só 1 saiu do lote.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO pedido (id_pedido, canal_venda, tipo_recebimento, status, id_cliente, id_funcionario) OVERRIDING SYSTEM VALUE
petly_club-# VALUES (92, 'loja_fisica', 'balcao', 'pago', 4, 2);
INSERT 0 1
petly_club=# INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario) VALUES (92, 2, 3, 29.90);
INSERT 0 1
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, id_lote, id_produto, id_funcionario, id_pedido)
petly_club-# VALUES ('venda', 1, 3, 2, 2, 92);
INSERT 0 1
-- consulta AU03 depois da inconsistência:
 id_pedido | id_produto | vendido | baixado
-----------+------------+---------+---------
        92 |          2 |       3 |       1
(1 row)

(ROLLBACK: a base volta ao estado original)
```

Decisão: Registrar a movimentação de venda que faltou, no lote correto (FEFO), ou investigar se o produto saiu sem registro.

[AU04] Reservas presas em pedidos encerrados

O que verifica: Procura reservas que continuam abertas em pedidos cancelados, entregues ou retirados. Uma reserva esquecida deixa o produto indisponível para venda sem que ele tenha saído do estoque.

Regra: RN21, RN22

SQL:

```
SELECT p.id_pedido, p.status, m.id_lote, m.id_produto,
       SUM(CASE WHEN m.tipo = 'reserva' THEN m.quantidade
                WHEN m.tipo IN ('liberacao_reserva', 'venda') THEN -m.quantidade
                ELSE 0 END) AS reserva_em_aberto
FROM movimentacao_estoque m
JOIN pedido p ON p.id_pedido = m.id_pedido
WHERE p.canal_venda = 'loja_virtual'
  AND p.status IN ('cancelado', 'entregue', 'retirado')
GROUP BY p.id_pedido, p.status, m.id_lote, m.id_produto
HAVING SUM(CASE WHEN m.tipo = 'reserva' THEN m.quantidade
                WHEN m.tipo IN ('liberacao_reserva', 'venda') THEN -m.quantidade
                ELSE 0 END) > 0
ORDER BY p.id_pedido;
```

Resultado na base do grupo: nenhuma linha.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
-- Reservas presas em pedidos encerrados (base do grupo)
 id_pedido | status | id_lote | id_produto | reserva_em_aberto
-----------+--------+---------+------------+-------------------
(0 rows)
```

Teste com erro proposital: O pedido foi cancelado, mas as 2 unidades reservadas nunca foram liberadas.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO pedido (id_pedido, canal_venda, tipo_recebimento, status, id_cliente) OVERRIDING SYSTEM VALUE
petly_club-# VALUES (93, 'loja_virtual', 'entrega', 'cancelado', 5);
INSERT 0 1
petly_club=# INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario) VALUES (93, 5, 2, 74.90);
INSERT 0 1
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, id_lote, id_produto, id_pedido)
petly_club-# VALUES ('reserva', 2, 7, 5, 93);
INSERT 0 1
-- consulta AU04 depois da inconsistência:
 id_pedido |  status   | id_lote | id_produto | reserva_em_aberto
-----------+-----------+---------+------------+-------------------
        93 | cancelado |       7 |          5 |                 2
(1 row)

(ROLLBACK: a base volta ao estado original)
```

Decisão: Registrar a liberação da reserva para devolver a quantidade ao disponível do catálogo (RF17).

[AU05] Pagamentos e contas a receber que não batem

O que verifica: Confere o financeiro de cada pagamento: autorizado tem de ter contas a receber que somam o seu valor; recusado ou pendente não pode ter conta; estornado só pode ter contas canceladas.

Regra: RN22, RN24

SQL:

```
SELECT g.id_pagamento, g.id_pedido, g.forma_pagamento, g.status, g.valor,
       COALESCE(SUM(c.valor_bruto), 0) AS soma_contas,
       COUNT(c.*) FILTER (WHERE c.status <> 'cancelada') AS contas_ativas
FROM pagamento g
LEFT JOIN conta_receber c ON c.id_pagamento = g.id_pagamento
GROUP BY g.id_pagamento, g.id_pedido, g.forma_pagamento, g.status, g.valor
HAVING (g.status = 'autorizado' AND COALESCE(SUM(c.valor_bruto), 0) <> g.valor)
    OR (g.status IN ('pendente', 'recusado') AND COUNT(c.*) > 0)
    OR (g.status = 'estornado' AND COUNT(c.*) FILTER (WHERE c.status <> 'cancelada') > 0)
ORDER BY g.id_pagamento;
```

Resultado na base do grupo: nenhuma linha.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
-- Pagamentos e contas a receber que não batem (base do grupo)
 id_pagamento | id_pedido | forma_pagamento | status | valor | soma_contas | contas_ativas
--------------+-----------+-----------------+--------+-------+-------------+---------------
(0 rows)
```

Teste com erro proposital: Um Pix autorizado que não gerou conta a receber.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO pagamento (id_pedido, forma_pagamento, valor, numero_parcelas, data_autorizacao, status)
petly_club-# VALUES (1, 'pix', 10.00, 1, '2026-10-20 11:00', 'autorizado');
INSERT 0 1
-- consulta AU05 depois da inconsistência:
 id_pagamento | id_pedido | forma_pagamento |   status   | valor | soma_contas | contas_ativas
--------------+-----------+-----------------+------------+-------+-------------+---------------
            9 |         1 | pix             | autorizado | 10.00 |           0 |             0
(1 row)

(ROLLBACK: a base volta ao estado original)
```

Decisão: Corrigir o lançamento antes do fechamento: um valor previsto a mais ou a menos distorce o fluxo de caixa (RF37).

[AU06] Compras com recebimento ou financeiro inconsistente

O que verifica: Compara o status de cada compra com o que realmente chegou (entradas por item) e o total da compra com a soma das parcelas a pagar ([TRG-06], [TRG-09]).

Regra: RN12, RN25

SQL:

```
WITH receb AS (
    SELECT r.id_compra,
           SUM(r.quantidade_comprada) AS comprado,
           SUM(LEAST(r.quantidade_recebida, r.quantidade_comprada)) AS recebido
    FROM v_item_compra_recebido r GROUP BY r.id_compra),
fin AS (SELECT id_compra, SUM(valor) AS parcelas FROM conta_pagar GROUP BY id_compra)
SELECT c.id_compra, c.status, r.comprado, r.recebido,
       t.valor_total, COALESCE(f.parcelas, 0) AS parcelas
FROM compra c
JOIN receb r ON r.id_compra = c.id_compra
JOIN v_compra_total t ON t.id_compra = c.id_compra
LEFT JOIN fin f ON f.id_compra = c.id_compra
WHERE c.status <> 'cancelada'
  AND (   (c.status = 'recebida' AND r.recebido < r.comprado)
       OR (c.status = 'recebida_parcialmente' AND (r.recebido = 0 OR r.recebido >= r.comprado))
       OR (c.status = 'solicitada' AND r.recebido > 0)
       OR COALESCE(f.parcelas, 0) <> t.valor_total)
ORDER BY c.id_compra;
```

Resultado na base do grupo: nenhuma linha.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
-- Compras com recebimento ou financeiro inconsistente (base do grupo)
 id_compra | status | comprado | recebido | valor_total | parcelas
-----------+--------+----------+----------+-------------+----------
(0 rows)
```

Teste com erro proposital: Uma compra marcada como 'recebida' sem nenhuma entrada e sem conta a pagar.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO compra (id_compra, data_compra, status, id_fornecedor) OVERRIDING SYSTEM VALUE
petly_club-# VALUES (90, '2026-10-26', 'recebida', 3);
INSERT 0 1
petly_club=# INSERT INTO item_compra (id_compra, id_produto, quantidade_comprada, custo_unitario) VALUES (90, 4, 10, 8.00);
INSERT 0 1
-- consulta AU06 depois da inconsistência:
 id_compra |  status  | comprado | recebido | valor_total | parcelas
-----------+----------+----------+----------+-------------+----------
        90 | recebida |       10 |        0 |       80.00 |        0
(1 row)

(ROLLBACK: a base volta ao estado original)
```

Decisão: Cobrar do fornecedor o que falta entregar ou corrigir as parcelas antes do vencimento.

[AU07] Lotes vencidos ainda com saldo

O que verifica: Procura lotes cuja validade já passou e que ainda têm unidades no estoque. Esses produtos não podem ser vendidos e precisam sair por perda.

Regra: RF12, RP09

SQL:

```
SELECT l.id_lote, pr.nome, l.codigo_lote, l.data_validade,
       CURRENT_DATE - l.data_validade AS dias_vencido,
       s.saldo_fisico
FROM lote l
JOIN produto pr ON pr.id_produto = l.id_produto
JOIN v_lote_saldo s ON s.id_lote = l.id_lote
WHERE l.data_validade < CURRENT_DATE
  AND s.saldo_fisico > 0
ORDER BY l.data_validade;
```

Resultado na base do grupo: nenhuma linha.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
-- Lotes vencidos ainda com saldo (base do grupo)
 id_lote | nome | codigo_lote | data_validade | dias_vencido | saldo_fisico
---------+------+-------------+---------------+--------------+--------------
(0 rows)
```

Teste com erro proposital: Um lote de ração para gatos que venceu em 15/10/2026 e ainda tem 4 unidades.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO lote (id_lote, codigo_lote, data_entrada, data_validade, id_produto) OVERRIDING SYSTEM VALUE
petly_club-# VALUES (90, 'LOTE-GAT-VENC', '2026-08-01', '2026-10-15', 5);
INSERT 0 1
petly_club=# INSERT INTO item_compra (id_compra, id_produto, quantidade_comprada, custo_unitario) VALUES (102, 5, 4, 40.00);
INSERT 0 1
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, data_hora, id_lote, id_produto, id_funcionario, id_compra)
petly_club-# VALUES ('entrada', 4, '2026-08-01 09:00', 90, 5, 3, 102);
INSERT 0 1
-- consulta AU07 depois da inconsistência:
 id_lote |           nome            |  codigo_lote  | data_validade | dias_vencido | saldo_fisico
---------+---------------------------+---------------+---------------+--------------+--------------
      90 | Racao Gatos Castrados 3kg | LOTE-GAT-VENC | 2026-10-15    |           12 |            4
(1 row)

(ROLLBACK: a base volta ao estado original)
```

Decisão: Retirar o lote da prateleira e registrar a perda com autorização da gerência (RP11).

[AU08] Perdas e ajustes autorizados por quem não é gerente

O que verifica: Confere quem autorizou cada perda e ajuste de estoque. O CHECK exige um autorizador, mas não sabe o cargo dele; a regra de que só a gerência autoriza fica nas permissões ([TRG-17]).

Regra: RP08, RP11

SQL:

```
SELECT m.id_movimentacao, m.data_hora, m.tipo, m.quantidade, m.motivo,
       a.nome AS autorizador, a.cargo
FROM movimentacao_estoque m
JOIN funcionario a ON a.id_funcionario = m.id_autorizador
WHERE m.tipo IN ('perda', 'ajuste_positivo', 'ajuste_negativo')
  AND a.cargo <> 'gerente'
ORDER BY m.data_hora;
```

Resultado na base do grupo: nenhuma linha.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
-- Perdas e ajustes autorizados por quem não é gerente (base do grupo)
 id_movimentacao | data_hora | tipo | quantidade | motivo | autorizador | cargo
-----------------+-----------+------+------------+--------+-------------+-------
(0 rows)
```

Teste com erro proposital: Uma perda autorizada pelo estoquista Roberto, que não é gerente.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO movimentacao_estoque (tipo, quantidade, motivo, id_lote, id_produto, id_funcionario, id_autorizador)
petly_club-# VALUES ('perda', 1, 'Embalagem rasgada', 2, 1, 4, 3);
INSERT 0 1
-- consulta AU08 depois da inconsistência:
 id_movimentacao |         data_hora          | tipo  | quantidade |      motivo       |   autorizador   |   cargo
-----------------+----------------------------+-------+------------+-------------------+-----------------+------------
              36 | 2026-10-27 23:51:34.025025 | perda |          1 | Embalagem rasgada | Roberto Martins | estoquista
(1 row)

(ROLLBACK: a base volta ao estado original)
```

Decisão: Revisar a perda com a gerência e corrigir as permissões de quem autorizou.

[AU09] Descontos sem autorização registrada

O que verifica: Lista os pedidos com desconto que não têm o gerente que autorizou. O limite de desconto é parâmetro da aplicação, então a consulta mostra todos para conferência.

Regra: RP05, RF32

SQL:

```
SELECT p.id_pedido, p.data_pedido, p.canal_venda, p.desconto,
       t.valor_itens, f.nome AS registrado_por
FROM pedido p
JOIN v_pedido_total t ON t.id_pedido = p.id_pedido
LEFT JOIN funcionario f ON f.id_funcionario = p.id_funcionario
WHERE p.desconto > 0
  AND p.id_autorizador_desconto IS NULL
ORDER BY p.desconto DESC;
```

Resultado na base do grupo: nenhuma linha.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
-- Descontos sem autorização registrada (base do grupo)
 id_pedido | data_pedido | canal_venda | desconto | valor_itens | registrado_por
-----------+-------------+-------------+----------+-------------+----------------
(0 rows)
```

Teste com erro proposital: Um desconto de R$ 20,00 lançado no pedido 1 sem autorizador.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# UPDATE pedido SET desconto = 20.00 WHERE id_pedido = 1;
petly_club=# UPDATE 1
-- consulta AU09 depois da inconsistência:
 id_pedido |     data_pedido     | canal_venda | desconto | valor_itens | registrado_por
-----------+---------------------+-------------+----------+-------------+-----------------
         1 | 2026-10-20 10:30:00 | loja_fisica |    20.00 |      399.70 | Ana Paula Souza
(1 row)

(ROLLBACK: a base volta ao estado original)
```

Decisão: Confirmar com a gerência se o desconto foi autorizado e, se não foi, tratar com o funcionário.

[AU10] Cadastros de clientes possivelmente duplicados

O que verifica: O CPF não se repete (UNIQUE), mas o mesmo cliente pode ter sido cadastrado duas vezes com CPF digitado errado. A consulta procura clientes diferentes com o mesmo e-mail ou o mesmo telefone.

Regra: RN18, RF03

SQL:

```
SELECT c1.id_cliente, c1.nome, c1.cpf,
       c2.id_cliente AS possivel_duplicado, c2.nome AS nome_duplicado, c2.cpf AS cpf_duplicado,
       CASE WHEN c1.email = c2.email THEN 'mesmo e-mail' ELSE 'mesmo telefone' END AS motivo
FROM cliente c1
JOIN cliente c2 ON c2.id_cliente > c1.id_cliente
               AND (c1.email = c2.email OR c1.telefone = c2.telefone)
ORDER BY c1.id_cliente;
```

Resultado na base do grupo: nenhuma linha.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
-- Cadastros de clientes possivelmente duplicados (base do grupo)
 id_cliente | nome | cpf | possivel_duplicado | nome_duplicado | cpf_duplicado | motivo
------------+------+-----+--------------------+----------------+---------------+--------
(0 rows)
```

Teste com erro proposital: A cliente Maria Souza cadastrada de novo, com dois dígitos do CPF trocados.

psql — petly_club (PostgreSQL 16) — executado em 27/10/2026

```
petly_club=# INSERT INTO cliente (nome, cpf, telefone, email)
petly_club-# VALUES ('Maria Souza', '12345678910', '11988887777', 'maria.souza@email.com');
INSERT 0 1
-- consulta AU10 depois da inconsistência:
 id_cliente |    nome     |     cpf     | possivel_duplicado | nome_duplicado | cpf_duplicado |    motivo
------------+-------------+-------------+--------------------+----------------+---------------+--------------
          1 | Maria Souza | 12345678901 |                  8 | Maria Souza    | 12345678910   | mesmo e-mail
(1 row)

(ROLLBACK: a base volta ao estado original)
```

Decisão: Confirmar com o cliente qual cadastro é o correto e unificar o histórico de compras no cadastro certo.

Uso no dia a dia

Estas consultas podem rodar todo dia, antes do fechamento do caixa (RF26), ou ser transformadas em visões e alertas no painel da gerência proposto na análise de BI. Qualquer linha retornada é um ponto de atenção para a gerência, o estoque ou o financeiro conferir.